

Presentation Script: Energy-Aware Hybrid Recommender Systems Across the User Lifecycle

1. Title (0:30)

Good morning, and thank you for being here. My name is Saina Amiri Moghadam, and today I'm presenting my bachelor's thesis, *Energy-Aware Hybrid Recommender Systems Across the User Lifecycle*, written at the Chair of Connected Mobility.

The diagram on this slide shows the purpose of my thesis. Most papers I came across, address the classic challenges of recommender system such as cold start, sparsity, accuracy and scalability but what is missing often is the sustainability of a recommender system and that is the central topic of my thesis.

→ next slide

2. Introduction (1:25)

Recommender systems decide much of what we see online, from the restaurants suggested to us to the films a streaming service offers.

Over the last decade, the models behind them have become much more expensive to run. Two studies by Spillo and Vente measured this directly. They found that more sophisticated models emit far more carbon, but the extra cost does not always buy a proportionate gain in recommendation quality. For example: on MovieLens, a simple nearest-neighbor method matched or beat a complex graph model that emits almost 700 times more carbon.

[\[click\]](#)

But there is a second point that these studies don't cover. A recommender is not trained once. After deployment, new users sign up, new items appear, and existing users change their preferences. So the model has to be updated, as shown by the diagram on the right.

[\[click\]](#)

Existing energy studies measure a single training run. The recurring cost of keeping a model current, and what that cost actually buys in quality, has not been measured. That is the gap this thesis addresses.

→ next slide

3. Central Research Question (0:30)

This leads to my central question: *What does it cost to keep a recommender up to date, and can a cheaper hybrid mechanism deliver comparable quality?*

Behind this are four more specific questions: 1. the carbon cost of staying current, 2. whether a content-based hybrid can help with new users, 3. how update frequency trades quality against emissions, 4. and how much all of this depends on the dataset.

→ next slide

4. LightGCN (1:15)

The model I use throughout is LightGCN, a graph-based collaborative recommender.

On the left, you see the idea. Users and items are nodes in a graph, and every interaction, for example a user reviewing a business, is an edge between them.

Each user and item has an embedding, a vector of numbers, in this case 64 numbers per user and per item. On the right is the core operation: in each layer, a node's embedding is updated as a normalized average of its neighbors' embeddings. So a user is described by the items they interacted with, and an item by the users who interacted with it.

To predict whether a user will like an item, we simply take the dot product of their two embeddings, and the items with the highest scores are recommended.

LightGCN is simple, but it still propagates over the whole graph, so its cost grows as the graph gets bigger. That makes frequent retraining expensive, which is exactly the situation I'm interested in.

→ next slide

5. Measuring Carbon Emissions (1:10)

To measure cost, I use CodeCarbon, an open-source Python library. My server has no hardware energy counters, so CodeCarbon estimates CPU and RAM energy from utilization.

[\[click\]](#)

Emissions are the product of two factors: the energy consumed, E , and the carbon intensity of the electricity grid, C .

[\[click\]](#)

Carbon intensity is a weighted average over the energy sources in the grid, so it depends on where the server is located. For Germany, that is 381 grams of CO₂-equivalent per kilowatt-hour.

One honest caveat: these are estimates, not physical measurements. But all strategies are measured in exactly the same way, so comparisons between them are valid.

I also don't just record one total per run. I track each phase separately. That breakdown will show us later where the cost actually comes from.

→ next slide

6. Evaluation Metrics (1:15)

To measure quality, I evaluate every batch as it arrives, before the model learns from it. For each user in the batch, the model scores all items, removes the ones the user has already interacted with, and recommends the top 10. ONE IMPORTANT definition is The relevant items which is defined as the items the user actually liked in that batch.

[\[click\]](#)

Precision@10 asks: of the 10 recommended items, how many were relevant?

[\[click\]](#)

Recall@10 asks the opposite: of all relevant items, how many did we recommend?

[\[click\]](#)

And NDCG@10 also looks at the order. A hit at rank 1 counts more than a hit at rank 10, and the score is normalized by the best possible ranking, so it lies between 0 and 1.

In most of my results, all three tell the same story, so I'll show Recall@10 throughout . Where precision or NDCG tell a different story, I'll point it out on that slide.

→ next slide

7. Comparison of Methods (1:10)

To address the two sources of quality decay, I use two mechanisms.

Incremental updating keeps users the model already knows up to date.

Content-based initialization improves performance for users the model was never trained on, using the content of the first items they interact with.

[\[click\]](#)

Neither one alone covers both sources of quality decay. So I combine them, and compare every option not only by recommendation quality, but by quality against carbon emissions.

IT is important to say here. Hybrid recommenders are not new. But the usual designs, such as weighted or switching hybrids, train a second model next to the first, which adds cost. They are also not designed with sustainability in mind: a survey of 76 hybrid systems found that their computational cost is rarely even reported, and none of the hybrids I reviewed is used to reduce or postpone retraining. My hybrid trains nothing extra, and it is designed specifically to keep the carbon cost of staying up to date low.

→ next slide

8. Methodology (1:15)

Here is how the experiments work.

I use two datasets, Yelp and MovieLens. Both are ordered by timestamp. The first 80 percent is used to train the base LightGCN model, and the remaining 20 percent is streamed in batches of 1,000 interactions to simulate a real time stream. Splitting along the global timeline guarantees the model never sees data from the future; I'll come back to this at the end.

[\[click\]](#)

I compare five strategies. No-update which frozen model right after training and is the lower bound. Incremental update runs 30 training epochs every 20 batches, starting from the current model. Full retraining rebuilds the model from scratch, which is the expensive upper bound. Then there is content-based initialization, and finally the combination of content-based initialization with incremental updates.

[\[click\]](#)

For every batch, I measure Recall@10, and emissions.

→ next slide

9. Content Signals: Location and Category (0:50)

Now let me zoom in on the content-based part, since it is the main new component. First, I need a way to describe items by their attributes. On Yelp, each business has a location as latitude and longitude. Raw coordinates are hard to compare, so I use geohash.

Geohash divides the map into rectangles and assigns each an alphabet and zoom into each rectangle and further divide them and repeats. As a result, nearby locations share the same prefix, as you see here, where all cells start with the same three characters. I use four-character geohashes, which is roughly a 39 by 20 kilometer area. This turns location into discrete buckets that can be counted and compared directly.

The second signal is simpler: each business's category, for example restaurant or bar. On MovieLens, which has no locations, I use genre and release decade instead.

→ next slide

10. Content-Based Initialization (1:45)

So how do I use these signals?

In no update and incremental, a new user gets the average of all existing embeddings, so everyone new looks the same.

But here, after training, I build a profile for every trained user, namely how often they interacted with items in each geohash bucket and each business category.

[\[click\]](#)

When a new user arrives, I compare their first interactions with these profiles. For each bucket, I take the smaller of the two counts and sum them up, the reason for this is that so a trained user can't look similar just because they have a huge count in one bucket. The similarity score weights location at 0.7 and category at 0.3, since I made the assumption that location is the stronger signal on Yelp. In this small example, user 3 overlaps most with the new user, then user 1.

[\[click\]](#)

Then I take the 20 most similar trained users and initialize the new user's embedding as a weighted average of theirs, with the normalized scores as weights.

This initialization happens only once per user. After that, the embedding is refined only by normal training updates if it is the combined strategies otherwise it stays at it is.

→ next slide

11. Yelp: No-Update vs. Incremental Update (1:15)

Now the results, starting with Yelp.

This plot shows Recall@10 over the stream. The thin lines are per batch, the bold lines a rolling

average of 30 batches, and the dashed vertical lines mark each incremental update.

Both strategies start at the same point, but they diverge almost immediately. The frozen model decays steadily, to roughly half its starting level. The incremental model improves by about 30 percent and then holds.

It is also worth noting that the frozen model doesn't fall at one specific point. It builds up throughout, as the stream moves further away from the data it was trained on. After about 120,000 interactions, the gap stays roughly constant.

On average, incremental updating improves the recall by 91 percent. It wins in about 95 percent of individual batches, and the difference is statistically significant.

So clearly, updating matters. But the question is why the frozen model decays.

→ next slide

12. Yelp: Active Users per Update Window (1:20)

To find out, I first looked at who is actually in the stream. This plot shows, for every update window of 20 batches, how many distinct users were active. Orange are new unique users, people we see for the very first time. Blue are returning users the model has already seen, either in training or in earlier batches.

On average, about 9 percent of the active users in a window appear for the first time, and you can see the orange part shrinking over the stream. That decline is partly an artifact of my 10-interaction filter: since newer users have had less time to reach ten interactions. Without the filter, arrivals stay much more stable; that plot is in the appendix.

But this plot understates one thing, since a user only counts as “new unique user” on their first appearance. If we Count everyone who was not in the training data, 41 percent of users in the stream are new, and they produce 45 percent of all interactions.

→ next slide

13. Yelp: Existing vs. New Users (1:15)

So almost half of the stream comes from users the model was never trained on. How well does the model serve them? To answer that, I split the evaluation into two groups: users the model was trained on, and new users.

The answer is: pretty bad

In this plot, the bottom line is new users. Their recall is about 70 percent below the overall average, and it stays at that level throughout. Existing users, the top line, perform above average but slowly decline over the stream, and the overall also decline similarly. The new-user gap, however, stays constant.

So on Yelp, a large part of the quality decay is a cold-start problem, not just drift. This diagnosis is what motivates the content-based initializer.

→ next slide

14. Yelp: No-Update vs. Content-Based Initialization (1:05)

Here is content-based initialization against no-update. Note that neither strategy retrains.

Content-init improves overall Recall@10 by 41 percent, again significant, winning in about 97 percent of batches. And unlike incremental updating, it helps from the very first batch, because new users get a better embedding the moment they appear.

Notice that the two lines rise and fall together. The gap doesn't grow over time, because neither model is ever retrained. It's a stable, constant improvement.

[\[click\]](#)

Split by group, existing users are unchanged, as expected, since only new users' embeddings differ. But the new-user line rises substantially. The gap between new users and the average shrinks from about 70 percent to about 11 percent. At some points, new users even reach the existing users' level, as the existing users' frozen embeddings become outdated.

→ next slide

15. Yelp: All Strategies (1:20)

Now all strategies together.

The combined strategy, content-based initialization plus incremental updates, is the best throughout. It starts ahead like content-init, and its lead grows over time like incremental updating.

You can also see that content-init leads early, before incremental updating has accumulated enough updates, and then incremental catches up and overtakes it. Incremental catches up because each update adds all interactions since the last update, from new and existing users, as real edges to the graph, and then refines the embeddings. Content-init, by contrast, always relies on the same frozen model, so its advantage doesn't compound.

Later in the stream the two incremental variants overlap, because fewer new users arrive.

[\[click\]](#)

Split by group, the combined strategy does something interesting. After about 80,000 interactions, new users actually perform better than existing users. The new users have gone from the worst-served group to slightly above average.

→ next slide

16. Comparison of Strategies (1:20)

Let me summarize the numbers. The first row is recall relative to the frozen baseline.

[\[click\]](#)

Incremental updates improves by 91 percent.

[\[click\]](#)

Content-based initialization by 41 percent, about half of that, but at the lowest added cost.

[\[click\]](#)

And the combination: by 106 percent, the largest gain.

[\[click\]](#)

The second row is the cost. Training the base model once emits about 1.8 kilograms of CO₂-equivalent. Running the whole stream costs between 5.2 and 7.6 percent of that, depending on the strategy. So even the most expensive strategy adds less than a tenth of the training cost.

Relative to the no update, incremental updating adds about 31 percent to the streaming cost, content-init about 18 percent, and the combination about 45 percent. Per unit of added cost, incremental updating buys the most recall.

Interestingly, the largest single cost is not the training steps at all, but growing the embedding tables as new users and items arrive. this is show in the graph in appendix

→ next slide

17. Yelp: Update Frequency vs. Emissions (1:30)

So far, updates happened every 20 batches, which was an arbitrary choice. Here I varied the update frequency of the combined strategy across ten settings, from every single batch to every 270 batches.

Each point is one setting: emissions on the x-axis and recall on the y-axis. The curve shows diminishing returns. Going from every 270 to every 180 batches gives about 6 percent more recall for only 4 percent more emissions. Every 16 batches gives 27 percent more recall for 20 percent more emissions, so quality and cost still grow together. But updating after every single batch gives 45 percent more recall at 250 percent more emissions.

To find the point where extra emissions stop paying off, I used knee detection. I draw a straight reference line between the cheapest and the most expensive run, shown in gray, and measure how far each run lies from it. The run furthest from the line is the knee, the point where the curve bends most strongly. The knee is around every 5 batches, with 5, 9 and 16 nearly equal. Beyond that, you pay a lot for very little.

→ next slide

18. MovieLens: Active Users per Update Window (1:10)

Now MovieLens, which tells a very different story. Let's start again with who is in the stream.

Compared to Yelp, a much larger share of the activity comes from new unique users, the orange part. But they almost stop arriving after about 100,000 interactions. Here, that is not caused by my filter. MovieLens-1M is already filtered by its creators: it only contains users with at least 20 ratings, all of whom joined in 2000, while the data runs until early 2003. So the later part of the stream is sparse.

The users also behave differently. They tend to rate many movies in one session when they sign up, backfilling their history. For a typical new user, 98 percent of their ratings arrive in their first batch, compared to 20 percent on Yelp. And 48 percent of new users never appear again; on Yelp, that is 12 percent.

→ next slide

19. MovieLens: All Strategies (0:50)

What does this mean for the strategies? Here is Recall@10. All five strategies are nearly indistinguishable for most of the stream, including full retraining. Only at the very end does the combined strategy pull slightly ahead.

The reason is what we just saw: by the time an update could help a user, that user is usually gone. There is simply nothing for the update to correct.

And this is expensive problem to ignore: full retraining costs about 790 times more per update than incremental updating, and about 7.6 times more over the whole lifecycle, without better recommendations.

→ next slide

20. MovieLens: Precision@10, All Strategies (1:15)

The backfilling we just saw also shows up in the metrics, so I want to show Precision@10 graph. While recall stayed roughly flat, precision looks completely different: it first rises, then drops sharply. That is exactly when new users stop arriving.

Why the difference? While new users are arriving, each one backfills most of their history in a single batch, so in that batch they have many relevant items at once. Precision always divides by 10, so more relevant items push precision up. NDCG behaves the same way. Recall divides by the number of relevant items instead, so the extra items appear on both sides of the fraction and largely cancel out. Once new users stop arriving, the backfilling stops, and precision and NDCG fall, while recall stays where it was.

Notice also that the drop hits all five strategies equally. It's a property of the data, not of any update strategy.

→ next slide

21. Pitfall: Per-User Split (1:10)

One lesson from this work concerns data preparation itself.

My first experiments used a per-user split, which is common in the literature; In one paper that looked at the data preparation about 12 percent of surveyed papers split by time. This plot shows those results. The frozen model barely decays, because the training data already contains information from the future. it also suppress the improvement of the strategies. Incremental updating gains only 11 percent, and incremental and content-init look roughly equal.

Compare this with the global timeline split from the results earlier, where the picture is completely different: the frozen model loses half its recall, incremental gains 91 percent, and it is clearly stronger than content-init. Content-init is also affected, but less: its gain drops from 41 to 17 percent. The leak removes the staleness that incremental updating targets, but not the cold-start gap. So a leaky split doesn't just inflate the numbers; it can change which method you conclude is best.

→ next slide

22. Conclusion (0:50)

To conclude: cheap update strategies recover most of the quality a frozen model loses, at a small fraction of the training cost, but only where the data actually has something to correct. On Yelp, keeping the model current cost between 6 and 7.6 percent of training and improved recall by 41 to 106 percent.

[\[click\]](#)

So my main takeaway is this: before choosing an update strategy, diagnose the source of decay. On Yelp, new users were a major cause, and a nearly free content-based fix made a big difference. On MovieLens, there was little decay to fix, and every update, especially full retraining, was wasted emissions.

→ next slide

23. Next Steps (1:10)

There are several directions for future work.

First, CI-LightGCN, a causal incremental version of LightGCN. Instead of retraining on the full graph, it reuses the stored embeddings from the previous model and only processes the new part of the graph. Its authors report retraining more than 30 times faster than full retraining while even exceeding its accuracy, but they measured time, not energy, so its actual carbon savings are still open.

[\[click\]](#)

Second, building the graph from a fixed time window, which would cap the size of the graph and therefore the cost of each update.

[\[click\]](#)

Third, streams with controlled, documented drift, so drift detection can be properly validated. My own drift detectors found no usable signal on Yelp.

[\[click\]](#)

And finally, more datasets with less strict filtering, where I expect content-based initialization to matter even more.

→ next slide

24. Acknowledgements (0:15)

I'd like to thank the Chair of Connected Mobility, my examiner, Prof. Jörg Ott, and my advisor, Prof. Wolfgang Wörndl.

Thank you for your attention. I'm happy to take your questions.

Backup: Likely Questions

- **Why no full retrain on Yelp?** Yelp's graph grows to about 2.5 million interactions and would need 25 retrains versus 8 on MovieLens, roughly nine times the work. That was not

affordable on the CPU-only server. → appendix carbon table

- **Why is absolute recall so low?** Each batch is scored on only 1,000 interactions against all items, and only the current batch counts as ground truth. Relative comparisons between strategies are what matter. → appendix Precision/NDCG slides
- **How reliable are CodeCarbon numbers?** They are estimates (no hardware counters), but every strategy is measured identically, so comparisons hold.
- **Were $\alpha = 0.7$, $k = 20$ and 30 epochs tuned?** No, they were fixed by inspection, so the reported gains are a lower bound.
- **Why does the number of new users drop over the Yelp stream?** It’s an artifact of the 10-interaction filter; without it, arrivals are stable. → appendix “New Unique Users per Update Window”
- **Where does the cost go?** Mostly into expanding the embedding tables, which RecBole requires to match the graph size. → appendix “Emissions by Phase”